

Avaliação de Performance

Calculando a acurácia do nosso modelo preditivo.



```
# Calcular acurácia
acuracia = accuracy_score(y_test, previsoes)
print("=" * 50)
print("📊 PERFORMANCE DO MODELO")
print("=" * 50)
print(f"Acurácia: {acuracia*100:.2f}%")
print(f"\nIsso significa que o modelo acertou")
print(f"{acuracia*100:.1f}% das previsões no conjunto de teste!")
print(f"\nTotal de previsões: {len(y_test)}")
print(f"Previsões corretas: {int(acuracia * len(y_test))}")
print(f"Previsões erradas: {len(y_test) - int(acuracia * len(y_test))}")
```



~80%

Acurácia Obtida

Modelo acertou 80% das previsões no conjunto de teste



Interpretação

Uma acurácia de ~80% indica que o modelo acertou 80% das previsões feitas.

A aceitação depende do contexto: diagnósticos médicos exigem >95%, enquanto recomendações de filmes podem aceitar 70%.

Além da acurácia

Avaliando precisão e revocação

Relatório completo

```
print("=" * 50)
print("📄 RELATÓRIO DE CLASSIFICAÇÃO")
print("=" * 50)
print(classification_report(
    y_test,
    previsoes,
    target_names=['Não Sobreviveu',
                  'Sobreviveu']
))
```

0.82

Precision

0.79

Recall

0.80

F1-Score

📌 Métricas Importantes:

- ✓ **Precision:** Das previsões positivas, quantas eram reais?
- ✓ **Recall:** Dos casos positivos reais, quantos foram encontrados?
- ✓ **F1-Score & Support:** Média harmônica entre as métricas e a quantidade total de casos reais analisados.

**Nota: Para o Titanic, buscamos um alto recall para 'Sobreviveu'.*



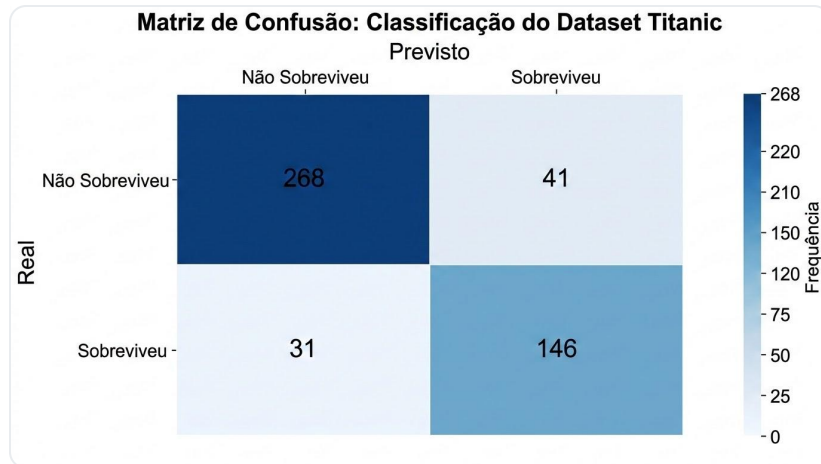
Matriz de Confusão

Visualizando os acertos e erros do modelo.

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Calcular matriz de confusão
matriz = confusion_matrix(y_test, previsoes)

# Visualizar Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(matriz, annot=True, fmt='d',
            cmap='Blues',
            xticklabels=['Não Sobreviveu', 'Sobreviveu'],
            yticklabels=['Não Sobreviveu', 'Sobreviveu'])
plt.title('Matriz de Confusão - Titanic')
plt.ylabel('Real')
plt.xlabel('Previsto')
plt.show()
```



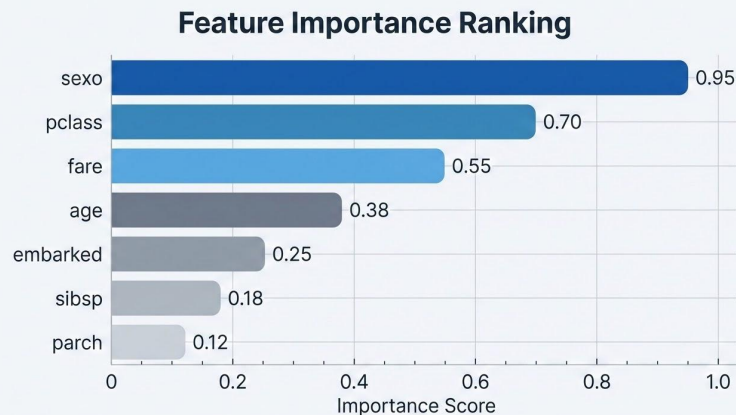
Resultado: Verdadeiros Positivos (146) e Verdadeiros Negativos (268).

- **Verdadeiros Positivos (Sobreviveu):** 146
- **Falsos Positivos / Falsos Negativos:** 41 / 31

Importância das Features

Identificando quais variáveis mais influenciaram o modelo

```
# Visualizar importância das features
plt.figure(figsize=(10, 6))
importancias_sorted = importancias.sort_values(
    'Importancia', ascending=True)
plt.barh(importancias_sorted['Feature'],
importancias_sorted['Importancia'],
color='skyblue', edgecolor='black')
plt.title('Importância das Features')
```



Insight Principal

O **sexo** é tipicamente a feature mais importante (mulheres tiveram prioridade na evacuação), seguido diretamente pela **classe social (pclass)** e a **tarifa paga (fare)**.



Modelagem de Dados

Aplicando o modelo a dados inéditos

```
# Criar dados de novo passageiro
# Mulher, 25 anos, primeira classe, embarcou em Cherbourg
novo_passenger = pd.DataFrame({
    'pclass': [1], 'sex_encoded': [0], 'age': [25],
    'sibsp': [0], 'parch': [0], 'fare': [100],
    'embark_encoded': [0]
})

# Fazer previsão
previsao = modelo.predict(novo_passenger)
probabilidade = modelo.predict_proba(novo_passenger)
print(f"Previsão: {previsao[0]}")
print(f"Probabilidade: {probabilidade[0][1]*100:.1f}%")
```

91.7%

Probabilidade de Sobrevivência

Resultado do Modelo

Nova Passageira:

Mulher, 25 anos, 1ª Classe

Previsão Final: Sobreviveu

Não sobrevivência estimada em 8.3%

Comparação de Cenários

Analizando o impacto de diferentes perfis na sobrevivência

```
# Testar múltiplos cenários
cenarios = pd.DataFrame({
    'pclass': [1, 3, 1, 3],
    'sex_encoded': [0, 0, 1, 1],
    'age': [25, 25, 25, 25],
    'fare': [100, 10, 100, 10],
})
previsoes = modelo.predict(cenarios)
probabs = modelo.predict_proba(cenarios)

# Exibição dos Resultados:
Cenário 1: Mulher, 25 anos, 1ª classe
  Previsão: Sobreviveu (Prob: 96.2%)
Cenário 4: Homem, 25 anos, 3ª classe
  Previsão: Não Sobreviveu (Prob: 12.4%)
```

Cenário Extremo A

Mulher, 1ª Classe

Idade: 25 anos • Tarifa: \$100

96.2%

Prob. Sobrevivência

✓ **Sobreviveu**

Forte impacto positivo da prioridade de embarque e classe social.

Cenário Extremo B

Homem, 3ª Classe

Idade: 25 anos • Tarifa: \$10

12.4%

Prob. Sobrevivência

✗ **Não Sobreviveu**

Baixa prioridade estrutural e localização das cabines periféricas.



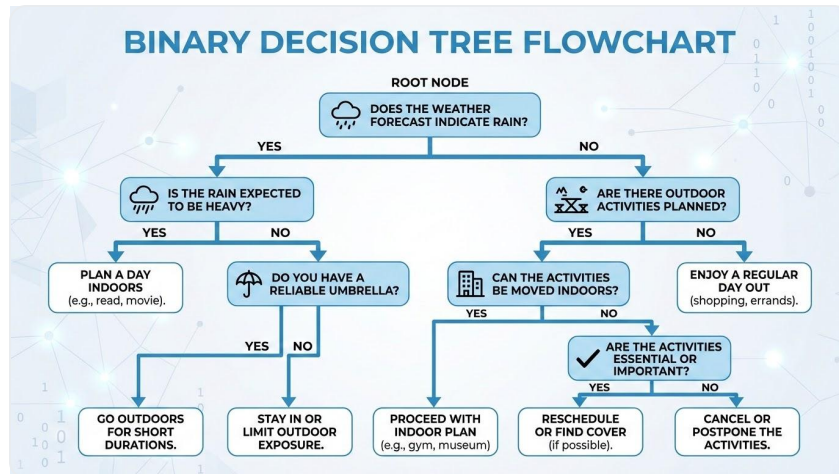
Insight Principal

Mulheres da primeira classe possuem a maior probabilidade de sobrevivência.

Árvore de Decisão

Entendendo a lógica interna de uma Árvore de Decisão

```
from sklearn.tree import plot_tree
# Criar modelo com uma única árvore
arvore_unica = RandomForestClassifier(
    n_estimators=1, max_depth=3,
    random_state=42
)
arvore_unica.fit(X_train, y_train)
# Plotar árvore
plt.figure(figsize=(15, 10))
plot_tree(arvore_unica.estimators_[0],
    feature_names=features,
    class_names=['Não Sobreviveu',
    'Sobreviveu'], filled=True)
```



Árvore de Decisão (simplificada)



Explicação

Cada nó representa uma decisão lógica (ex: 'sexo <= 0.5?'). As folhas finais representam a previsão do modelo.



Exportando o Modelo

Salvando o modelo treinado e encoders para ambiente de produção

```
import joblib

# Salvar modelo e encoders

joblib.dump(modelo, 'modelo_titanic.pkl')
joblib.dump(encoder_sex, 'encoder_sex.pkl')
joblib.dump(encoder_embark,
'encoder_embark.pkl')

print("✅ Modelo salvo!")
print("Arquivos criados:")
- modelo_titanic.pkl (modelo)
- encoder_sex.pkl (sexo)
- encoder_embark.pkl (embarque)
```

- ✅ **modelo_titanic.pkl** (modelo treinado)
- ✅ **encoder_sex.pkl** (encoder de sexo)
- ✅ **encoder_embark.pkl** (encoder de embarque)

Importância da Exportação

Salvar o modelo permite sua imediata aplicação em cenários do mundo real (APIs, plataformas web e aplicativos móveis), eliminando por completo a necessidade e o custo computacional de um re-treinamento contínuo do algoritmo.



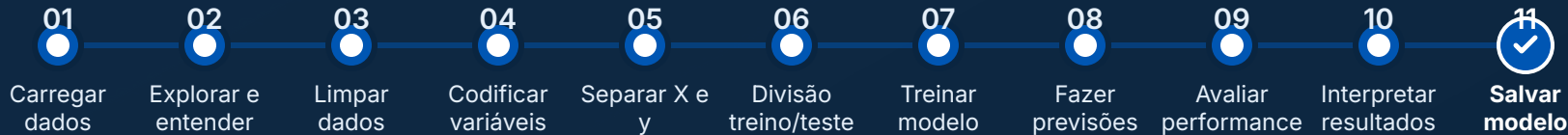
Interface de Predição

Criando uma interface simples para utilizar o modelo salvo

```
def prever_sobrevivencia(sexo, idade, classe, tarifa, porto):  
    """ Prevê sobrevivência no Titanic """  
    sexo_cod = 0 if sexo.lower() == 'female' else 1  
    porto_cod = encoder_embark.transform([porto])[0]  
    passageiro = pd.DataFrame({  
        'pclass': [classe], 'sex_encoded': [sexo_cod], 'age': [idade],  
        'sibsp': [0], 'parch': [0], 'fare': [tarifa], 'embark_encoded': [porto_cod]  
    })  
    previsao = modelo.predict(passageiro)[0]  
    prob = modelo.predict_proba(passageiro)[0][1]  
    resultado = "Sobreviveu" if previsao == 1 else "Não Sobreviveu"  
    print(f"\nResultado: {resultado} | Probabilidade: {prob*100:.1f}%")  
    return resultado, prob  
  
# Testar a função com dados de exemplo  
prever_sobrevivencia('female', 25, 1, 100, 'Cherbourg')
```



Ciclo de Vida Padrão de um Projeto Preditivo



Celebre!

Você construiu seu primeiro modelo preditivo seguindo o fluxo padrão da indústria!



Desafio Prático

Previsão de Inadimplência Bancária



Visão Geral do Desafio



Objetivo: Construir um modelo para prever se um cliente de banco não pagará um empréstimo.



Dataset: "Bank Marketing Dataset" (UCI)



Problema: Classificação binária (0 ou 1)



Tempo: 20 minutos



Dica: Use o código do Titanic como base!

Passos a seguir

- 01 Carregar dataset
- 02 Explorar dados
- 03 Tratar ausentes
- 04 Codificar variáveis
- 05 Dividir treino/teste
- 06 Treinar modelo
- 07 Avaliar acurácia
- 08 Interpretar resultados



Passo 1: Carga de Dados

Iniciando o desafio prático com dados reais.

```
# Dataset de marketing bancário
# (previsão de inadimplência)
url =
'https://raw.githubusercontent.com/ ... /bank-full.csv'
df_bank = pd.read_csv(url, sep=';')

print("✅ Dataset carregado!")
print(f"Total de clientes: {len(df_bank)}")
print(f"Colunas: {len(df_bank.columns)}")

print("\nPrimeiras linhas:")
print(df_bank.head())

print("\nColunas disponíveis:")
print(df_bank.columns.tolist())
```

Colunas Importantes

- **Perfil Demográfico & Financeiro:** age, job, marital, balance
- **Indicadores de Risco:** housing, loan
- **Variável Alvo (y):** subscrição confirmada (yes / no) representando o target final para classificação

Estrutura do Conjunto de Dados

- **Total de registros:** carregamento completo via Pandas preservando as relações individuais de cada cliente
- **Consistência:** validação estrutural inicial feita exibindo as primeiras linhas do DataFrame estruturado



Exploração do Dataset

Conhecendo a estrutura e distribuição dos dados bancários

```
# Estatísticas descritivas do dataset
print("Distribuição da target (y):")
df_bank['y'].value_counts()
tx_sub = (df_bank['y'] == 'yes').mean()

# Estatísticas de Idade e Saldo
print(f"Média Idade:
{df_bank['age'].mean():.1f}")
print(f"Min/Max: {df_bank['age'].min()} /
{df_bank['age'].max()}")
print(f"Média Saldo:
{df_bank['balance'].mean():.0f}")

# Identificação de dados faltantes
df_bank.isnull().sum()
```

Taxa de Subscrição (y)
Conversão de campanhas

11.7%

Atributos do Dataset Bancário

- **Idade (age):** Média: 40.9 anos (Min: 18 | Max: 95)
- **Saldo (balance):** Média: 1.362 USD
- ✓ **Valores Ausentes:** Nenhum campo nulo detectado



Preparação de Dados

Separando variáveis e criando os conjuntos de treino e teste.



```
# Selecionar features relevantes
features_bank = ['age', 'balance', 'duration', 'campaign',
                 'pdays']

# Codificar target: yes=1, no=0
df_bank['y_encoded'] = (df_bank['y'] == 'yes').astype(int)

# Separar X e y
X_bank = df_bank[features_bank]
y_bank = df_bank['y_encoded']

# Dividir treino/teste
X_train_bank, X_test_bank, y_train_bank, y_test_bank =
train_test_split(
    X_bank, y_bank, test_size=0.2, random_state=42,
    stratify=y_bank
)

print("✅ Dados preparados!")
```

CONJUNTO DE TREINO (80%)

36168 clientes

Usado para o ajuste de parâmetros e aprendizado do modelo.

CONJUNTO DE TESTE (20%)

9043 clientes

Utilizado para avaliação final e validação da capacidade de generalização.



Aplicando o Random Forest

Treinamento e avaliação do modelo no problema bancário

```
# Criar e treinar modelo
modelo_bank = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    max_depth=5
)
modelo_bank.fit(X_train_bank, y_train_bank)
print("✅ Modelo treinado!")

# Fazer previsões
preveesoes_bank = modelo_bank.predict(X_test_bank)

# Calcular acurácia
acuracia_bank = accuracy_score(y_test_bank, preveesoes_bank)
print(f"Acurácia: {acuracia_bank*100:.2f}%")
print(classification_report(y_test_bank, preveesoes_bank,
    target_names=['Não Subscriveu', 'Subscriveu']))
```

90.15%

ACURÁCIA DE TESTE

Classes Avaliadas

- Não Subscriveu
- Subscriveu

classification_report()



Estruturando os achados do modelo

Template de documentação e métricas para o caso Bank Marketing

RESULTADOS DO MODELO - BANK MARKETING

****Acurácia:**** XX.XX%

****O que isso significa:**** O modelo acertou XX de cada 100 previsões.

****Precision para 'Subscriveu':**** XX%

Das previsões de subscrição, XX% estavam corretas

****Recall para 'Subscriveu':**** XX%

Dos clientes que realmente subscreveram, o modelo encontrou XX%

****F1-Score:**** XX (Média harmônica entre precision e recall)

****Insights:****

1. [Observação sobre performance]
2. [Observação sobre limitações]
3. [Sugestão de melhorias]



5:00

Minutos Restantes

Tempo de Preenchimento

Use este tempo para analisar as métricas geradas pelo seu modelo e preencher a documentação estruturada ao lado.



Para quem terminou cedo

Prevendo preços de imóveis com Regressão Linear

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Carregar dados e dividir
california = fetch_california_housing()
X_cal = pd.DataFrame(california.data,
                     columns=california.feature_names)
y_cal = california.target
X_train_cal, X_test_cal, y_train_cal, y_test_cal = \
    train_test_split(X_cal, y_cal, test_size=0.2, random_state=42)

# Criar modelo e prever
modelo_reg = LinearRegression()
modelo_reg.fit(X_train_cal, y_train_cal)
previsoes_reg = modelo_reg.predict(X_test_cal)

print(f"MSE: {mean_squared_error(y_test_cal, previsoes_reg):.2f}")
print(f"R²: {r2_score(y_test_cal, previsoes_reg):.2f}")
```

Avaliação de Regressão

- **MSE (Erro Quadrático Médio)**

Mede a média dos erros elevados ao quadrado. Penaliza grandes desvios, sendo essencial para identificar previsões muito distantes do real.

- **R² (Coeficiente de Determinação)**

Indica a proporção da variância dos preços que é explicável pelas variáveis do modelo. Quanto mais próximo de 1.0, melhor o ajuste.



Praticando a Comunicação de Resultados

Formato da Apresentação (1 min por dupla)

- 1 Dataset utilizado
- 2 Tipo de problema (Classificação/Regressão)
- 3 Acurácia ou métrica principal
- 4 Feature mais importante
- 5 Um insight interessante
- 6 Dificuldade encontrada



Exemplo Prático

*"Usei o dataset **Titanic**, problema de **classificação**, acurácia de **82%**, feature mais importante foi **sexo**, insight: **mulheres de 1ª classe tinham 90% de chance de sobreviver**, dificuldade: **entender matriz de confusão**."*



O que não fazer ao construir modelos preditivos

Evite erros metodológicos clássicos na modelagem de ciência de dados

⚠ Erros Críticos de Iniciantes

Data Leakage

Usar informações do futuro para prever o passado.

Overfitting

Modelo complexo que decora o treino, mas não generaliza.

Ignorar NaNs

Algoritmos e modelos matemáticos falham com valores ausentes.

⚙ Outros Desvios & Boas Práticas

Outros Pontos de Atenção

- **Não dividir dados:** Avaliar no mesmo dado usado no treino.
- **Ignorar desbalanceamento:** Proporções muito desiguais de classes.
- **Esquecer codificação:** Não tratar adequadamente variáveis categóricas.

✓ Soluções Recomendadas

Dividir os dados antes do processamento, usar validação cruzada e documentar cada etapa rigorosamente.



Checklist de Boas Práticas

Garantindo a qualidade e confiabilidade do seu modelo

- ✓ Dividir dados em treino/teste ANTES do processamento
- ✓ Usar `random_state` para reprodutibilidade
- ✓ Tratar valores ausentes adequadamente
- ✓ Codificar variáveis categóricas
- ✓ Verificar desbalanceamento de classes
- ✓ Usar múltiplas métricas (não apenas acurácia)
- ✓ Interpretar importância das features
- ✓ Testar com dados novos/hipotéticos
- ✓ Documentar e salvar o modelo



Lembrete de Ouro

"Garbage in, garbage out."

O modelo de Machine Learning é estritamente tão bom quanto a qualidade e integridade dos dados fornecidos.



Ampliando o Leque de Opções

Indo além do Random Forest com múltiplos algoritmos do Scikit-Learn

```
from sklearn.linear_model import LogisticRegression
modelo_lr = LogisticRegression()

from sklearn.tree import DecisionTreeClassifier
modelo_dt = DecisionTreeClassifier()

from sklearn.svm import SVC
modelo_svm = SVC()

from sklearn.neighbors import KNeighborsClassifier
modelo_knn = KNeighborsClassifier()

from sklearn.ensemble import GradientBoostingClassifier
modelo_gb = GradientBoostingClassifier()
```

Diferenças Conceituais Básicas

- **Regressão Logística & SVM:** Buscam limites de decisão lineares ou complexos (com kernels) para separar as classes de forma eficiente.
- **Árvores de Decisão & KNN:** Abordagens baseadas em regras de decisão estruturadas ou similaridade espacial por proximidade métrica.
- **Gradient Boosting:** Método de ensemble sequencial que foca na correção sistemática de erros cometidos pelos estimadores anteriores.



Dica de Implementação

Cada algoritmo possui pontos fortes e fracos específicos em relação ao tipo de dado. Teste diferentes opções e compare rigorosamente as métricas de validação!

Configurações pré-treino

Tunagem de Hiperparâmetros no Random Forest



```
modelo = RandomForestClassifier(  
    n_estimators=100, # nº árvores (mais=melhor)  
    max_depth=5, # profundidade (evita overfit)  
    min_samples_split=2, # min para dividir nó  
    min_samples_leaf=1, # min amostras na folha  
    random_state=42 # reprodutibilidade  
)
```

VALOR BAIXO (Ex: max_depth=2)

Underfitting: O modelo fica muito simples e não consegue capturar as relações complexas dos dados, resultando em menor acurácia no treino e teste.

VALOR ALTO (Ex: max_depth=20)

Overfitting: As árvores crescem demais e memorizam ruídos do treino. O modelo apresenta excelente acurácia no treino, mas falha com novos dados.

⚙️ Como escolher os valores?

Experimente diferentes valores (ex: n_estimators=[50, 100, 200]) e compare a acurácia. Técnicas avançadas como **Grid Search** automatizam esse processo (veremos em detalhes na aula 9).

Validação Cruzada (5 folds)

Evitando resultados baseados na 'sorte' da divisão dos dados.

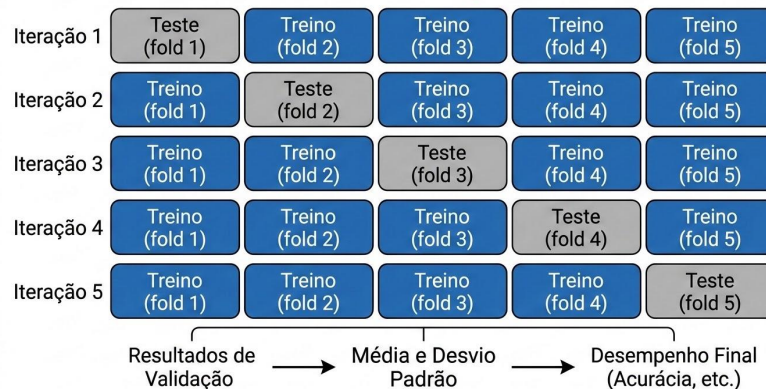
```
from sklearn.model_selection import cross_val_score
# Validação cruzada com 5 folds
scores = cross_val_score(modelo, X, y, cv=5,
                          scoring='accuracy')
print("✅ Validação Cruzada (5 folds)")
print(f"Scores: {scores}")
print(f"Média: {scores.mean()*100:.2f}%")
print(f"Desvio padrão: {scores.std()*100:.2f}%")
print(f"Intervalo: {scores.min()*100:.2f}% - {scores.max()*100:.2f}%")
```



Por que usar?

Testa o modelo em múltiplas divisões, fornecendo uma estimativa muito mais confiável da performance real.

5-Fold Cross-Validation: Diagrama Acadêmico



Esquema de rotação de partições de treino e teste do método K-Fold Cross Validation.



Template de Documentação

Garantindo que outros possam entender e reproduzir seu modelo.

```
# Projeto: Previsão de [TARGET]
## Autor: [Nome] | Data: [Data]

### 1. Descrição do Problema
[O que estamos tentando prever?]

### 2. Dataset
- Fonte: [URL] | Tamanho: [X linhas, Y colunas] | Target: [coluna]

### 3. Metodologia
- Algoritmo: [nome] | Features: [lista] | Split: [80/20]

### 4. Resultados
- Acurácia: XX% | Precision: XX% | Recall: XX%

### 5. Insights
1. [Insight 1] | 2. [Insight 2]

### 6. Próximos Passos
- [Melhoria 1]
```


Avalie seu progresso na atividade prática.

Checklist da Atividade e Autoavaliação

Lista de Verificação

Carreguei o dataset
Explorei dados
Tratei ausentes
Codifiquei variáveis
Separei features/target
Dividi treino/teste
Treinei modelo
Fiz previsões
Calculei acurácia
Gerei relatório
Interpretei resultados
Documentei insights
Salvei notebook



Autoavaliação de Desempenho

EXCELENTE

11-13 itens

Progresso ideal esperado

MUITO BOM

8-10 itens

Ótimo nível de entrega

Recomendações de Revisão:

- **5-7 itens:** Bom, revise os conceitos.
- **<5 itens:** Procure o suporte do professor.

Recapitulando a Aula

Fundamentos e conceitos essenciais de inteligência artificial



Conceitos-Chave

- **Inteligência Artificial:** Campo amplo de máquinas inteligentes.
- **Machine Learning:** Subcampo que aprende padrões a partir de dados.
- **Deep Learning:** Uso de redes neurais profundas.



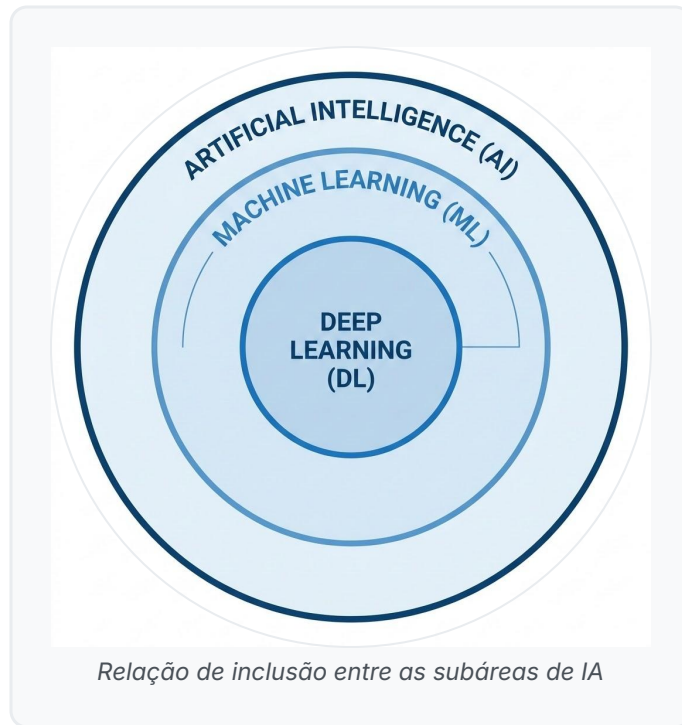
Tipos de ML

- **Supervisionado:** Dados rotulados (Classificação/Regressão).
- **Não-supervisionado:** Sem rótulos (Clusterização).
- **Reforço:** Baseado em recompensas (Jogos/Robótica).



Fluxo de Machine Learning

Dados → Limpeza → Features → Treino → Previsão → Avaliação



O roteiro da nossa jornada de aprendizado

Preview das Próximas Aulas



Cada aula constrói sobre a anterior!

Onde buscar mais conhecimento

Recursos Recomendados



Documentação & Vídeos



Scikit-learn.org

Excelente tutorial oficial para aprendizado prático.



Pandas.pydata.org

Guia essencial para manipulação e análise de dados.



Cursos & Vídeos

Kaggle Learn ("Intro to ML"), Coursera (Andrew Ng), DataCamp, StatQuest (Josh Starmer) e Corey Schafer.



LIVROS CLÁSSICOS

Hands-On Machine Learning

Aurélien Géron —
Excelente foco em
prática e aplicação de
ponta a ponta.

Introduction to Statistical Learning

James et al. — A base
teórica essencial e
indispensável para
algoritmos.



PRÁTICA REAL

Kaggle Competitions

Plataforma excelente
para praticar com
datasets reais do
mercado.

Titanic Dataset

O melhor e clássico
ponto de partida para
qualquer iniciante em
Data Science.

Ajude a melhorar as próximas aulas



Formulário Rápido (3 minutos)

[\[Link do Google Forms/Mentimeter\]](#)

Tópicos Avaliados:

- 1. Nível de dificuldade percebido.
- 2. Parte mais útil da aula.
- 3. Parte mais difícil (Teoria, Código, Métricas).
- 4. Ritmo da aula.
- 5. Confiança para continuar.
- 6. Sugestões abertas.



Por que o feedback importa?

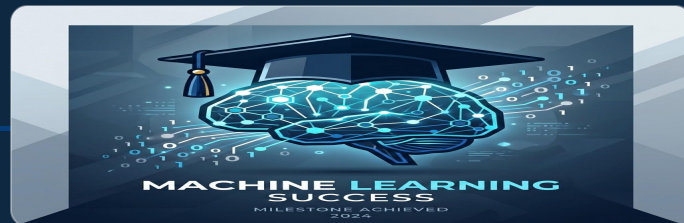
Ele ajusta o ritmo e foca nas suas reais necessidades de aprendizado!

Incrível!

Você construiu seu primeiro modelo preditivo funcional!

 **Mensagem Final:** Hoje você deu um passo gigante, saindo da teoria para a prática real. Lembre-se: todo expert foi iniciante um dia. Continue praticando e experimentando!

"O melhor modelo é aquele que você constrói e entende."



17 PRÓXIMA AULA

- **Tema:** Coleta de Dados - APIs.
- **Trazer:** Notebook salvo e dúvidas anotadas.



TAREFA DO DIA

Revisar o código e testar com um dataset diferente.